TileLang CUDA Z3：用一条 A+B 流水理解调度与自动调优

CUDA Z3 A+B latency II
buffer versions flat Task/Stage CUDA GDN

一、两种搜索：让一次执行正确，让调参少走弯路

1. 从一段已经很熟悉的 CUDA 计算开始

TileLang Python GPU CUDA NVIDIA GPU
CUDA

`C A B` CUDA **kernel** GPU **block**
A B block
buffer

tile
buffer **tile** **流水**

buffer
`A B`

tile
自动调优 autotuning **AutoTuner**
候选配置 **profiling**

CUDA

对比项	kernel 内调度	外层自动调优

对比项	kernel 内调度	外层自动调优
	kernel	

2. Z3 在两个位置扮演的角色

Z3

- **SAT**
- **UNSAT**
- **unknown**

SAT CUDA UNSAT
unknown

kernel “ ”” ”
” block ”

Z3 **成本模型**

AutoTuner
profiling

3. 为什么搜索加速和 kernel 加速要分开说

stock guided 9 guided
3 **guided**
kernel kernel

搜索墙钟时间 profiling **kernel latency** kernel 20% 20%

oracle “
” oracle guided

guided 3 3 3

guided

4. 一条连续例子和两类实验

A+B 4 2

1 SAT

2

CUDA buffer

“ ”

GDN **GDN** gated delta rule

二、一条 A+B 流水：从四个任务到最小 II

1. 把计算切成四个可讨论的边界

A B fragment

C **fragment** buffer

```
for   in
          A                    # T0: global -> shared
          B                    # T1: global -> shared
    for   in                   # T2: vector add

                     C         # T3: 写回 global
```

T0 T1 T2 T3

CUDA Z3

T0 T1 latency 4 T2 T3 latency 2 II 1 A+B kernel

2. latency 与 II：等待时间和接件节奏

latency T0 1 latency 4 5 " "

II initiation interval T0 II 1 1 4

latency II latency 4 II 4 II 1 latency 1

II ii_i II

3. 数据依赖决定“何时可读”

u v L_{uv}

$$start_v \geq start_u + L_{uv}.$$

v u v

T0→T2 T1→T2 T2→T3 4 4 2

- T0→T2 $6 - 1 = 5 \geq 4$ A
- T1→T2 $6 - 2 = 4 \geq 4$ B
- T2→T3 $8 - 6 = 2 \geq 2$

footprint **footprint**

4. 资源约束决定“何时能再接一件”

T0 T1 O_{01}

T0 T1 T0

91 T0 latency

T0 0 T1 1 T2 5 T3 7 9

makespan

---

6 T2 T1 8 T3

T2

T0 T1 T1 0 T0 1 T0 T1 T0

ii

6. 最小 II 搜索：为什么答案是 2

II 1

$$II_{hi} = \max\left(\sum_i latency_i,\ \sum_i ii_i\right) + 1.$$

" latency " " 1 12 4

1 13

II SAT UNSAT

1. $II = 7$ SAT
2. $II = 4$ SAT
3. $II = 2$ SAT
4. $II = 1$ UNSAT 2

$II = 1$ 0 T0 T1 ii 1

UNSAT

$II = 2$ latency k_i T2 3

2 2 2

fallback **fallback** II

UNSAT unknown

7. buffer versions：用存储换跨迭代距离

buffer **buffer versions**

ring 0 0 1 1

ring A+B buffer variables A B ring multi-async A+B

RAW read after write d RAW

$$start_v - start_u \geq L_{uv} - d \cdot II.$$

$d = 0$ 3 RAW $d \geq 1$ v d u II

buffer b

$$start_v - start_u \geq latency - II \cdot v_b.$$

v_b $II \cdot v_b$

T0 latency 4 1 2 $II = 2$ A RAW $2 - 1 = 1$ latency 4 $2 - 1 \geq 4 - 2$ $1 \geq 2$ $1 \geq 0$

T2 6

$$\sum_{b \in group} v_b \cdot size_b \leq C_{group}.$$

buffer C_{group} A B 512 B 1536 B 2 1 1536 B 2048 B

8. 用手算检查求解结果

Z3

- 5 4 2 latency
- 1 II 2
- start $k \cdot II + r$
- II=1 2
-

CUDA

buffer ring

三、把同一调度放进 CUDA：周期估算与 flat Task/Stage

1. 整数周期从哪里来

latency ii

SM120 compute capability 12.0 CUDA RTX 5090 Laptop GPU

微基准

$$completion = base + \left\lceil \frac{bytes}{bandwidth} \right\rceil + descriptor,$$

$$access_ii = \max\left(\left\lceil \frac{bytes}{bandwidth} \right\rceil, min_ii \right) + issue.$$

$base$ $bytes/bandwidth$ $descriptor$

2 128 B 64 B/ 1
$2 + \lceil 128/64 \rceil + 1 = 5$ 1 1 ii $\max(2, 1) + 1 = 3$
latency 5 ii 3

P Q

$$latency_{P+Q} = \max(P.latency,\ P.ii + Q.latency),$$

$$II_{P+Q} = P.ii + Q.ii.$$

P P Q Q

measured-only
fallback

2. flat Task：把源码边界交给调度器

IR intermediate representation IR
flat Task/Stage flat
Task Stage

body
body footprint **执行域**
Task

latency ii latency ii latency ii
" "

body
annotation

```
for   in
    with
                A
    with
                B
    with
        for   in

    with
                            C
# 手工分层时，在完整 Task 外组合 T.Stage(s)。
```

with body

Z3 IR **pass** stock CUDA pass

Task Stage Stage

stage 0 stage stage

stage

Task Stage k_i stage

$start = k \cdot II + r$ stage

3. 从 start/II 到 CUDA 能表达的阶段和顺序

CUDA

start

$$k_i = \left\lfloor \frac{start_i}{II} \right\rfloor, \qquad stage_i = k_i - \min_j(k_j).$$

0

II=2 T0 0 T1 1 T2 T3 3 4

$$residue_i = start_i - k_i \cdot II.$$

residue stage

residue T2 T3 0

拓扑顺序

" p " order "

"

$$inverse_order[physical_order[p]] = p.$$

2 0 0 order 2 " " "

"

buffer

$$cuda_depth \geq \max\left(num_stages,\ \max_i(stage_i) + 1,\ \max_b(v_b)\right).$$

stage

5 buffer 3 3

k_i r_i buffer

versions " "

4. 为什么物化后仍要做数值验收

CUDA A B A+B Task

CUDA PyTorch PyTorch

flat Stage

SAT

CPU IR

Z3 Task/Stage CUDA kernel profiling

四、求解可行之后：编译器必须接住的三类语义

1. SAT 与可执行代码之间的间隙

CUDA “v u” block u v

GDN

2. 两个 owner 的交接：正向到达还不够

owner warp group **warp group** warp warp CUDA 32 owner

owner-ring owner P C **role** CTA-local buffer ring CTA-local block P C

```
写入 A
C 读取 A
下一轮 必须等 C 释放，形成反向交接
```

P→C C P C→P **lifetime** ” ” ”

1 2 0 1 II

$$II \geq h_{P\to C} + h_{C\to P} = 1 + 2 = 3.$$

P C P $II = 3$ P 0 C 1 P 3 C P 2 $II = 2$ 1

II=3 residue 0 1 acquire commit wait release ring 1 2 3

owner role block ring stage-zero epoch **epoch** ring owner owner epoch

owner exclusion 3

3. 标量快照：消费者需要旧值，重放定义却读到新值

标量 IR **Bind** **pure**

copy producer stage 0 consumer stage 1 Bind consumer

```
A
B
B
下一轮 可能先覆盖 ，再执行
```

stage k consumer

consumer **clone** clone producer producer

3 clone 4 II 31 producer Bind clone consumer 0 30 31 33 clone 31 latency 2 consumer 33

consumer source clone clone

buffer **别名**

flat pure scalar Bind
producer clone

4. 异步等待：两个 producer stage 需要等谁

" " **异步拷贝**
CUDA producer consumer

commit group commit
N N

multi-async II 2
producer 0 1 consumer 3 4 producer buffer

```
A
B
对 A、B 所需的来源分别 ，再计算 C A B
写回 C
```

consumer

producer stage commit group
A B wait
N N

12 multi-async ring A B A+B
buffer variables 3/2
producer stage wait
consumer ring

1 4 5 12 1 4 5
12 ring drain **drain**

consumer buffer wait consumer buffer producer commit group

5. 三个案例共同改变了什么

owner-ring scalar clone multi-async commit group “ ” “ ” IR GDN

五、从候选到证据：五臂搜索、GDN 与模型的失误

1. 外层问题从一份候选清单开始

kernel tile

R raw space U profiling S $S \subseteq U \subseteq R$ legal U unknown stock-raw R

UNSAT unknown U “ ” “ ” UNSAT

GDN forward DV D 16 32 64 128 **动态共享内存** kernel block opt-in block 101376 B

$$B_{fwd} = 38912 + 576 \times block_DV.$$

38912 B DV 48128 57344 75776 112640 B
128 112640 B 101376 B R 4 U 3 stock-raw
guided

GDN backward prepare_h
归约
9 prepare_h

$$B_{prepare_h} = 57344 + 18432 \times stages.$$

18432 B 3 112640 B group 6

stage group

2. 成本模型怎样给保留候选排序

″ ″ ″ ″ c c
predicted_ms latency

kernel launch **launch** GPU kernel
launch launch phase **phase**

block ″ ″ SM **SM** streaming
multiprocessor GPU block SM block
block CUDA active-warp occupancy
warp warp waves block
waves

$$waves = \left\lceil \frac{block_count}{SM_count \times occupancy_ctas_per_sm} \right\rceil.$$

block block
block

phase

$$t_{overlap} = \begin{cases} \sum_r t_r, & stages = 1, \\ \max_r t_r, & stages \geq 2. \end{cases}$$

t_r r

phase launch predicted_ms

6 4 2 ms 12 ms 6 ms

6 ms kernel bank conflict

bank conflict

measured-only

unknown

U start II buffer ring

predicted_ms kernel

3. 五臂为什么能拆开收益来源

arm arm kernel

top-k k k

arm	实际选择方式	它帮助回答的问题
stock-raw		
z3-legal-exhaustive		
z3-cm-topk	k	
budget-raw	k	
budget-random	k	

guided " "

guided stock

S A stock AutoTuner

CUDA event profiling CUDA event GPU

GPU profile

搜索 wall guided

wall

arm cold cold Python

warmup 10 50 early stop **warmup**

early stop arm

winner candidate-median arm
cold latency

7 ABBA ABBA A B B A
50 warmup 200 L2 CUDA-event L2 GPU
arm “
”

4. GDN T8192：主结果怎样得到

GDN FlashQLA BF16 **BF16** 16
T=8192 forward backward
float64 relative-L2 0.02 relative-L2

held-out T4096 validation
T8192 held-out

forward top-2 backward top-3 cold
compile/profile wall speedup stock wall arm wall
confirmed latency

方向与 arm	compile / profile	wall（秒）	搜索加速	confirmed（ms）
fwd stock-raw	4 / 3	34.102	1.000×	0.308992
fwd legal-exhaustive	3 / 3	26.239	1.311×	0.308992
fwd guided top-2	2 / 2	19.410	1.757×	0.308992
fwd budget-raw	2 / 2	19.362	1.765×	0.308992
fwd budget-random	2 / 2	20.060	1.704×	0.308992
bwd stock-raw	9 / 6	62.690	1.000×	4.230792
bwd legal-exhaustive	6 / 6	56.734	1.105×	4.230792
bwd guided top-3	3 / 3	49.599	1.258×	4.230792
bwd budget-raw	3 / 3	49.813	1.253×	4.379024
bwd budget-random	3 / 3	55.322	1.139×	4.233904

forward guided profile 3 2
43.1% arm `D` confirmed latency 1.765×
guided 1.757×

backward guided profile 9 6 3 3
20.5% guided stage 2 group 8 4.230792 ms budget-raw
stage 1 group 8 latency 4.379024 ms budget-random stage 2 group 32 guided

guided budget-raw 0.962739
budget-random 0.999626 1
kernel 1

搜索过程 forward backward guided kernel
1.757× 1.258× GDN kernel owner-ring scalar clone multi-async

5. 排名质量：总体相关不等于保住第一名

winner Spearman Kendall
1

forward 1 backward 0.882735 0.787726
top-3 guided

backward stage 2 group 16 stage 2 group 8
group 16 top-3 top-1

oracle rank
oracle rank 16 top-10

6. 经典算子：搜索很快，仍可能漏掉最优

cold top-10 winner **工作负载** workload
GEMM K Conv2D stride
tail causal FlashAttention
GDN

guided/ 1 guided
latency GEMM Conv FlashAttention
stock-raw

工作负载	guided 搜索加速	guided（ms）	质量参照（ms）	guided / 质量参照	实测最优预测名次
GEMM	26.404×	1.472920	1.448576	1.017253	32–33 / 240

工作负载	guided 搜索加速	guided（ms）	质量参照（ms）	guided / 质量参照	实测最优预测名次
K GEMM	26.092×	0.428928	0.410432	1.039685	120 / 240
Conv2D stride tail	13.218×	0.016960	0.013088	1.295844	16 / 120
causal FlashAttention	4.321×	0.144040	0.140096	1.021014	27 / 36

guided top-10 GEMM 1.73% K 3.97% Conv 29.58% FlashAttention 2.10%
FlashAttention 2.10%

FlashAttention 54 36 36 30 stock-raw

Conv Spearman 0.80170
16 top-10 FlashAttention Spearman -0.39058
guided

GEMM guided K Conv
FlashAttention winner workload top-k

7. 失败记录怎样约束最终结论

T4096 validation modal-winner backward
guided/control 1.000624 " " candidate-median
T8192 held-out

GDN backward stage 1 group 8
1000 backward

no-cache TileLang CUDA warm
AutoTuner T4096 no-cache stock/guided forward
1.843× backward 1.276× warm 1.039× 1.165×
profile T8192 cold

workload
winner oracle rank

六、带走什么，怎样复现

1. 用五个问题重建整篇文章

latency 4 ii 1

II 2 8 II $start = k \cdot II + r$
T3 start 8 $k = 4$ 8

SAT CUDA owner
commit group

UNSAT
top-k

GDN 1.757× 1.258× profiling
guided kernel latency

2. 机制应该怎样使用

Z3 flat Task body
Task Stage start II
II

AutoTuner unknown
top-k
wall latency

workload k
Conv 29.58%

GPU RTX 5090 Laptop / SM120
kernel predicted_ms

3. 文章自身的最小检查

CPU

TileLang GPU

```
D EB EC DE B
```

KaTeX

HTML PDF ZIP LaTeX KaTeX LaTeX

Markdown

4. 内部调度的设备复现

compute capability 12.0 SM120 GPU TileLang CUDA Toolkit

Python compute capability 12.0 GPU CUDA Toolkit

“CUDA 12.0”

A+B Task Stage flat scalar clone multi-async

/multi-async start II 2 clone source

owner-ring

owner role skip

skip

GDN

5. GDN T8192 搜索与确认

repetition arm

forward 0

D E

repetition 1 2 warmup 10 repeat 50

forward top-2 backward top-3 GPU

root/output cold

best latency

winner forward

backward

wall latency

SM120

附录：证据索引与复现入口

1. 调度与设备正确性

SAT

- A+B Task/Stage scalar clone multi-async SM120 multi-async CPU multi-async-host-v2.txt
- owner role ring owner-sm120-v6.txt
- latency-model-tests.txt
- owner owner clone multi-async guard

flat

2. 硬件与标定

- hardware-sm120.json calibration-sm120.json
- calibration-summary.json
- EAD E
-

3. GDN held-out 与确认

- cold summary.csv oracle rank prediction-rank.json protocol-audit.json
- confirmed-summary.csv gdn-paired-differences.csv
- forward shared-memory prepare_h shared-memory
- validation T4096 backward 1000
- no-cache summary warm summary

A E cold

4. 经典算子与预测失误

- EAD E
- cold summary.csv top-k runs.csv
- winner confirmed-summary.csv confirmation-comparisons.csv

CSV raw legal selected compile attempt profile FlashAttention

5. 已接受捕获包

capture-acceptance-v1.json capture-manifest.json
CSV/JSON

- stock/guided capture metadata crop metadata analysis.json summary.csv GPU stock.log guided.log
- GDN capture metadata crop metadata analysis.json confirmation-comparisons.csv
-
- GEMM winner resource-usage.txt metadata.json kernel.sass static shared

ZIP
HTML